Хорошо, теперь как ни с роанно вернемся к lstm.

```
kernel void lstm_recurrent_forward(
    global const float
        *XW_in, // Готовая проекция X*W + bias: [Batch, SeqLen, 4 * HiddenSize]
    global const float *U,     // [HiddenSize, 4 * HiddenSize]
    global float *H_out,       // [Batch, SeqLen, HiddenSize]
    global float *workspace,  // [SeqLen, Batch, 5 * HiddenSize]
    global float *c_curr_buf, // [Batch, HiddenSize]
    global float *h_curr_buf, // [Batch, HiddenSize]
    const int batch_size, const int seq_len, const int hidden_size) {
  int b = get_group_id(0);
  int lid = get_local_id(1);
  int wg_size = get_local_size(1);

  if (b >= batch_size)
    return;

  global float *c_curr_ptr = c_curr_buf + (b * hidden_size);
  global float *h_curr_ptr = h_curr_buf + (b * hidden_size);

  for (int hid = lid; hid < hidden_size; hid += wg_size) {
    c_curr_ptr[hid] = 0.0f;
    h_curr_ptr[hid] = 0.0f;
  }
  barrier(CLK_GLOBAL_MEM_FENCE);

  int total_gates = 4 * hidden_size;
  int ws_stride = 5 * hidden_size;
  int vec_hidden = (hidden_size / 4) * 4;

  for (int t = 0; t < seq_len; t++) {
    // XW_in имеет форму [Batch, SeqLen, 4 * HiddenSize]
    int xw_offset = (b * seq_len + t) * total_gates;
    int ws_offset = (t * batch_size + b) * ws_stride;
    int h_out_offset = (b * seq_len + t) * hidden_size;

    // 1. Только рекуррентное умножение H_{t-1} * U + готовый XW_in
    for (int hid = lid; hid < hidden_size; hid += wg_size) {
      for (int g = 0; g < 4; g++) {
        int gate_idx = g * hidden_size + hid;
        float val = XW_in[xw_offset + gate_idx];

        int h = 0;
        for (; h <= hidden_size - 4; h += 4) {
          float4 h_vec = vload4(0, h_curr_ptr + h);
          float4 u_vec = (float4)(U[(h + 0) * total_gates + gate_idx],
                                  U[(h + 1) * total_gates + gate_idx],
                                  U[(h + 2) * total_gates + gate_idx],
                                  U[(h + 3) * total_gates + gate_idx]);
          val += dot(h_vec, u_vec);
        }
        for (; h < hidden_size; h++) {
```

```
        val += h_curr_ptr[h] * U[h * total_gates + gate_idx];
      }

      workspace[ws_offset + gate_idx] = val;
    }
  }

  barrier(CLK_GLOBAL_MEM_FENCE);

  // 2. Векторизованные активации по 4 нейрона за итерацию
  for (int hid = lid * 4; hid < vec_hidden; hid += wg_size * 4) {
    float4 i_gate =
        sigmoid4(vload4(0, workspace + ws_offset + 0 * hidden_size + hid));
    float4 f_gate =
        sigmoid4(vload4(0, workspace + ws_offset + 1 * hidden_size + hid));
    float4 g_gate =
        tanh(vload4(0, workspace + ws_offset + 2 * hidden_size + hid));
    float4 o_gate =
        sigmoid4(vload4(0, workspace + ws_offset + 3 * hidden_size + hid));

    float4 c_prev = vload4(0, c_curr_ptr + hid);
    float4 c_val = f_gate * c_prev + i_gate * g_gate;

    vstore4(c_val, 0, c_curr_ptr + hid);
    vstore4(c_val, 0, workspace + ws_offset + 4 * hidden_size + hid);

    float4 h_val = o_gate * tanh(c_val);
    vstore4(h_val, 0, h_curr_ptr + hid);
    vstore4(h_val, 0, H_out + h_out_offset + hid);
  }

  // Хвост, если hidden_size не кратен 4
  for (int hid = vec_hidden + lid; hid < hidden_size; hid += wg_size) {
    float i_gate = sigmoid(workspace[ws_offset + 0 * hidden_size + hid]);
    float f_gate = sigmoid(workspace[ws_offset + 1 * hidden_size + hid]);
    float g_gate = tanh(workspace[ws_offset + 2 * hidden_size + hid]);
    float o_gate = sigmoid(workspace[ws_offset + 3 * hidden_size + hid]);

    float c_val = f_gate * c_curr_ptr[hid] + i_gate * g_gate;
    c_curr_ptr[hid] = c_val;
    workspace[ws_offset + 4 * hidden_size + hid] = c_val;

    float h_val = o_gate * tanh(c_val);
    h_curr_ptr[hid] = h_val;
    H_out[h_out_offset + hid] = h_val;
  }

  barrier(CLK_GLOBAL_MEM_FENCE);
 }
}
```

Здесь мы с векторизацией немного премудрили и мне кажеться нам стоит немного изменить код, разбавив векторизацию полноценнмы использованием осей.

```
int b = get_group_id(0);
int lid = get_local_id(1);
int wg_size = get_local_size(1);

if (b >= batch_size)
  return;

global float *c_curr_ptr = c_curr_buf + (b * hidden_size);
global float *h_curr_ptr = h_curr_buf + (b * hidden_size);

for (int hid = lid; hid < hidden_size; hid += wg_size) {
  c_curr_ptr[hid] = 0.0f;
  h_curr_ptr[hid] = 0.0f;
}
barrier(CLK_GLOBAL_MEM_FENCE);
```

локальные потокии (cuida.threads) у gPU обычно не больше 2000 штук . hidden_size у нас в среднем 200-500 , но в может уходить и за 1000 . Но даже при 500 у нас есть интересный цикл :

```
    // 1. Только рекуррентное умножение H_{t-1} * U + готовый XW_in
    for (int hid = lid; hid < hidden_size; hid += wg_size) {
      for (int g = 0; g < 4; g++) {
        int gate_idx = g * hidden_size + hid;
        float val = XW_in[xw_offset + gate_idx];
```

Здесь мы умножаем hidden_size*4 а это не влезит в размер локального группы даже при 500 ! Вместо этого нам следует задействовать подноценный get_global_id по 3 осям (3 для батча так как он у нас самый маленький) .
Тогда этот цикл

```
for (int hid = lid; hid < hidden_size; hid += wg_size) {
    for (int g = 0; g < 4; g++) {
      int gate_idx = g * hidden_size + hid;
      float val = XW_in[xw_offset + gate_idx];

      int h = 0;
      for (; h <= hidden_size - 4; h += 4) {
        float4 h_vec = vload4(0, h_curr_ptr + h);
        float4 u_vec = (float4)(U[(h + 0) * total_gates + gate_idx],
                                U[(h + 1) * total_gates + gate_idx],
                                U[(h + 2) * total_gates + gate_idx],
                                U[(h + 3) * total_gates + gate_idx]);
        val += dot(h_vec, u_vec);
      }
      for (; h < hidden_size; h++) {
        val += h_curr_ptr[h] * U[h * total_gates + gate_idx];
      }
```

```
    workspace[ws_offset + gate_idx] = val;
  }
 }
```

схлопниться в относительно небольшой код (можно запустить ось с размерами hidden_size*4) а в ядре извлечь от из идентификатора значение g и hid!

```
// ========================================================================
// ЭТАП 2: Облегченный рекуррентный цикл по времени (H_{t-1} * U + XW)
// ========================================================================
int maxWg = (int)_computeCtx.MAX_WORK_GROUP_SIZE;
int wgSizeY = Math.Min(256, maxWg);
if (hiddenSize < wgSizeY)
{
    wgSizeY = 32;
    while (wgSizeY < hiddenSize && wgSizeY < maxWg) wgSizeY *= 2;
}

int argIdx2 = 0;
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(xwBuf);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(u.DataBuffer);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(hOut.DataBuffer);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(workspace);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(cCurrBuf);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(hCurrBuf);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(batchSize);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(seqLen);
_kernelLstmRecurrentForward.Arguments[argIdx2++].SetValue(hiddenSize);

activeQueue.EnqueueNDRangeKernel(
    _kernelLstmRecurrentForward,
    new IntPtr[] { (IntPtr)batchSize, (IntPtr)wgSizeY },
    new nint[] { (IntPtr)1, (IntPtr)wgSizeY }
);
```